# Penetration Test Report

**Security Assessment of Futurus**

Guillermo Pamplona
guigapamplona@gmail.com

April 13 2026

# Contents


**1. Engagement Overview** **3**
- 1.1. Assessment Objectives . . . 3
- 1.2. Scope . . . 3
- 1.3. Threat Model and Assumptions . . . 3
- 1.4. Methodology . . . 3
- 1.5. Risk Rating Approach . . . 4
- 1.6. Limitations . . . 4

**2. Executive Summary** **5**
- 2.1. Assessment Outcome . . . 5
- 2.2. Key Findings at a Glance . . . 6
- 2.3. Business Impact . . . 6
- 2.4. Immediate Priorities . . . 6

**3. Findings details** **7**
- 3.1. LOW Outdated JavaScript Libraries with Known Vulnerabilities . . . 7
- 3.2. LOW Weak Content Security Policy Due to script-src 'unsafe-inline' . . . 12
- 3.3. INFO Content Security Policy Hardening Gaps . . . 16

**4. Remediation and Closure** **20**
- 4.1. Assessment Artifacts and Cleanup . . . 20
- 4.2. Mitigation and Patching Guidance . . . 20
- 4.3. Retest Recommendation . . . 20
- 4.4. Confidential Information Handling . . . 20

**A. Appendices** **21**
- A.1. Summary of Passing Tests . . . 21

# 1. Engagement Overview

This report documents the results of the security assessment performed against the in-scope assets identified by the client. It is written to support both technical remediation and executive decision-making, with findings prioritized according to business risk and exploitability.

## 1.1. Assessment Objectives

The objectives of this engagement were to identify exploitable vulnerabilities, validate realistic attack paths, measure potential business impact, and provide practical remediation guidance. The assessment also evaluated whether weaknesses could expose sensitive data, enable unauthorized access, or reduce the confidentiality, integrity, or availability of business systems.

## 1.2. Scope

| | |
|---|---|
| **In-scope targets** | https://futurus.com.br |
| **Assessment window** | 13/04/26 to 13/04/26 |
| **Engagement type** | External |
| **Testing depth** | Black box |

## 1.3. Threat Model and Assumptions

The assessment assumed a motivated attacker with the level of access defined in the rules of engagement. This involved a black box penetration test, where no information besides the target domain was given.

## 1.4. Methodology

Testing should align with current industry practice and be tailored to the engagement type. A typical workflow includes:

1. Reconnaissance and asset validation.
2. Enumeration of services, technologies, trust relationships, and attack surface.
3. Vulnerability identification through manual testing and focused tooling.

4. Exploitation to confirm impact while remaining within agreed safety limits.
5. Post-exploitation validation, privilege analysis, and lateral movement assessment where authorized.
6. Documentation of evidence, affected assets, and remediation guidance.

## 1.5. Risk Rating Approach

Each finding in this report includes a CVSS v3.1 base score to provide a consistent technical severity rating. Final remediation priority should also consider business context, asset criticality, data sensitivity, exploit maturity, exposure, and the availability of compensating controls.

## 1.6. Limitations

This report reflects the state of the assessed environment during the testing window only. Security posture may change as systems are patched, configurations are modified, assets are added or removed, or new vulnerabilities become publicly known after the assessment date.

# 2. Executive Summary

The follow-up penetration test evaluated Futurus external security posture from an external attacker perspective during the assessment period in April 2026. The objective of this engagement was to validate the remediation status of findings identified during the previous assessment and to identify any additional security weaknesses observable within the agreed external scope.

Testing confirmed that the previously reported findings reviewed during this follow-up had been successfully remediated. Based on the activities performed, no evidence was obtained that the issues identified in the prior assessment remained exploitable within the retest scope. This indicates a meaningful improvement in the organization's external security posture and demonstrates effective remediation of the originally identified risks.

In addition to validating remediation, the assessment identified several lower-risk security hardening opportunities, including improvements to Content Security Policy configuration and updates to outdated client-side libraries. These items did not result in confirmed exploitation during testing, but addressing them would further strengthen browser-side protections, reduce exposure to known dependency-related issues, and improve the overall resilience of the application.

## 2.1. Assessment Outcome

The follow-up assessment confirmed that the findings selected for retesting had been effectively remediated and were no longer reproducible during the engagement. No unauthorized access, sensitive data exposure, or privilege escalation was achieved from the previously reported issues within the retest scope.

Additional observations were limited to lower-risk hardening opportunities, including CSP improvements and client-side library updates. These items should be addressed as part of ongoing security maintenance, but they did not alter the overall conclusion that the external security posture has improved since the previous assessment.

## 2.2. Key Findings at a Glance

**LOW** **Outdated JavaScript Libraries with Known Vulnerabilities** **3.7**

CVSS:3.1/AV:N/AC:H/PR:N/UI:N/S:U/C:L/I:N/A:N
The application uses outdated versions of jQuery and jQuery UI associated with publicly disclosed vulnerabilities.

**LOW** **Weak Content Security Policy Due to script-src 'unsafe-inline'** **3.7**

CVSS:3.1/AV:N/AC:H/PR:N/UI:N/S:U/C:L/I:N/A:N
The application's Content Security Policy permits inline JavaScript execution through the use of 'unsafe-inline' in the script-src directive.

**INFO** **Content Security Policy Hardening Gaps** **0**

CVSS:3.1/AV:N/AC:L/PR:L/UI:N/S:U/C:N/I:N/A:N
Additional CSP weaknesses were identified, including style-src 'unsafe-inline', an overly broad or insufficiently restricted img-src, and the absence of an explicit form-action directive.

## 2.3. Business Impact

Remediation of the previously reported findings has reduced the risk of unauthorized access, data exposure, and related business impact within the tested scope. The remaining items are low-risk hardening opportunities that do not currently represent confirmed exploitable conditions.

## 2.4. Immediate Priorities

No immediate critical action is required based on this follow-up. Priority should be given to routine hardening improvements such as strengthening CSP and updating legacy client-side libraries.

# 3. Findings details

## 3.1. LOW Outdated JavaScript Libraries with Known Vulnerabilities

CVSS v3.1 Base Score 3.7

| Attack Vector | Attack Complexity | Privileges Required | User Interaction |
|---|---|---|---|
| Network | Low | None | None |
| Adjacent | High | Low | Required |
| Local | | High | |
| Physical | | | |

| Scope | Confidentiality Imp. | Integrity Impact | Availability Impact |
|---|---|---|---|
| Unchanged | None | None | None |
| Changed | Low | Low | Low |
| | High | High | High |

**Summary** The application uses outdated versions of jQuery and jQuery UI associated with publicly disclosed vulnerabilities. During testing, this was identified as a dependency hygiene issue rather than a confirmed exploitable client-side vulnerability.

**Impact** If the application uses vulnerable library functions with attacker-controlled input, these components could increase exposure to client-side attacks such as XSS or unsafe object manipulation. No working exploit path was confirmed during this assessment.

**Proof of concept** Use OWASP ZAP in order to crawl the website and look for general vulnerabilities.

- Define the scope:

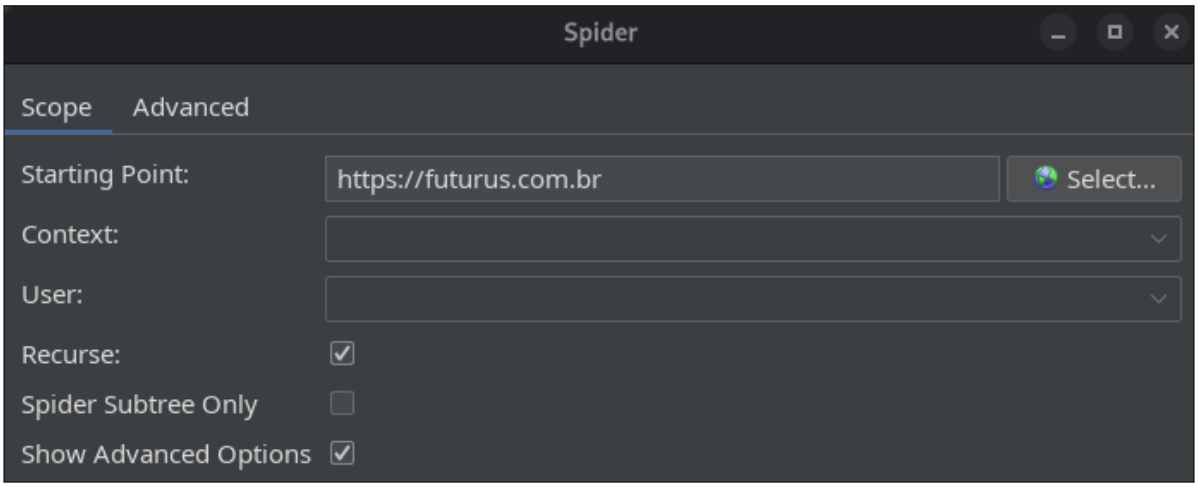



**Figure 3.1.:** Scope of the spider

- Set the following advanced settings:

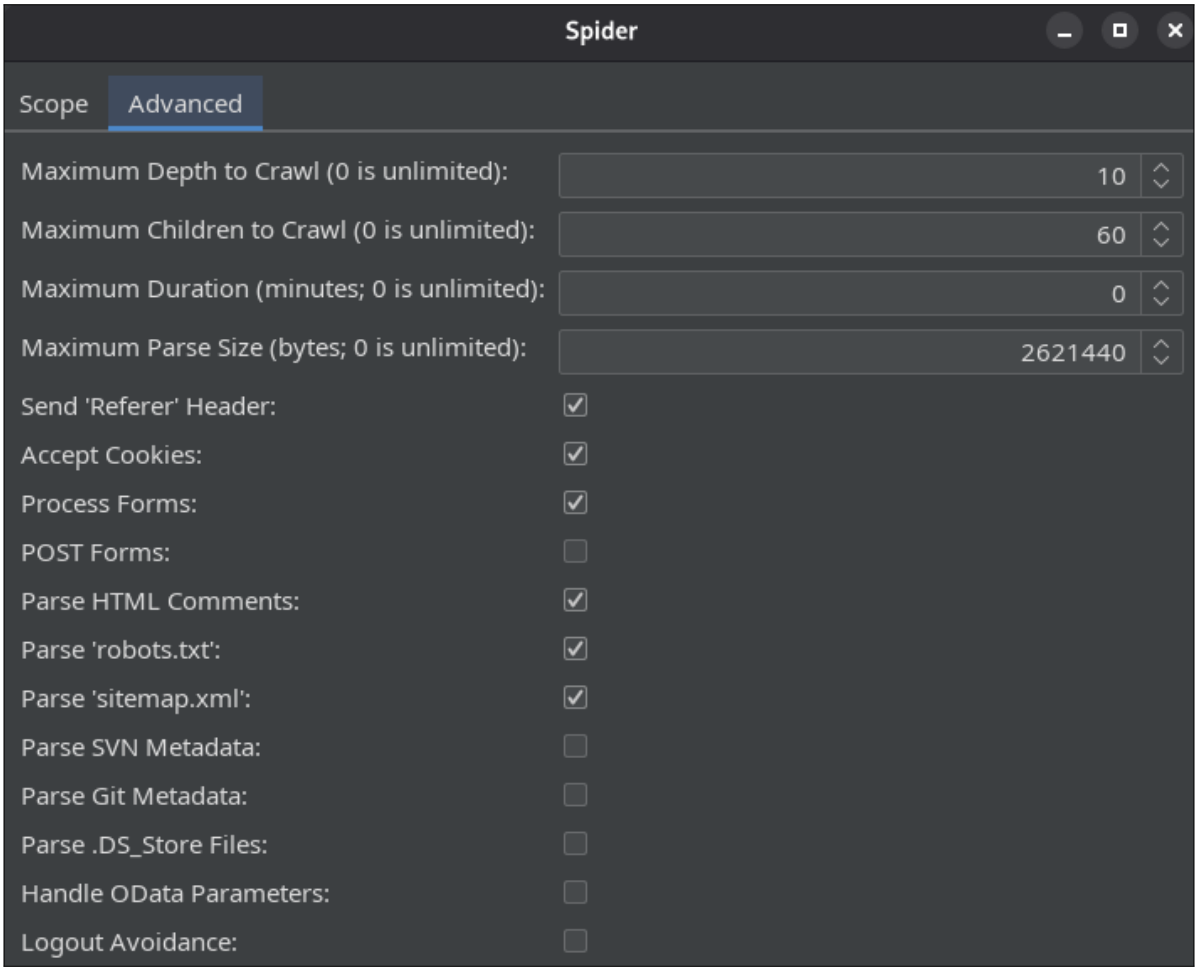



**Figure 3.2.:** Advanced settings to configure the spider

- Look the information gathered by the spider:

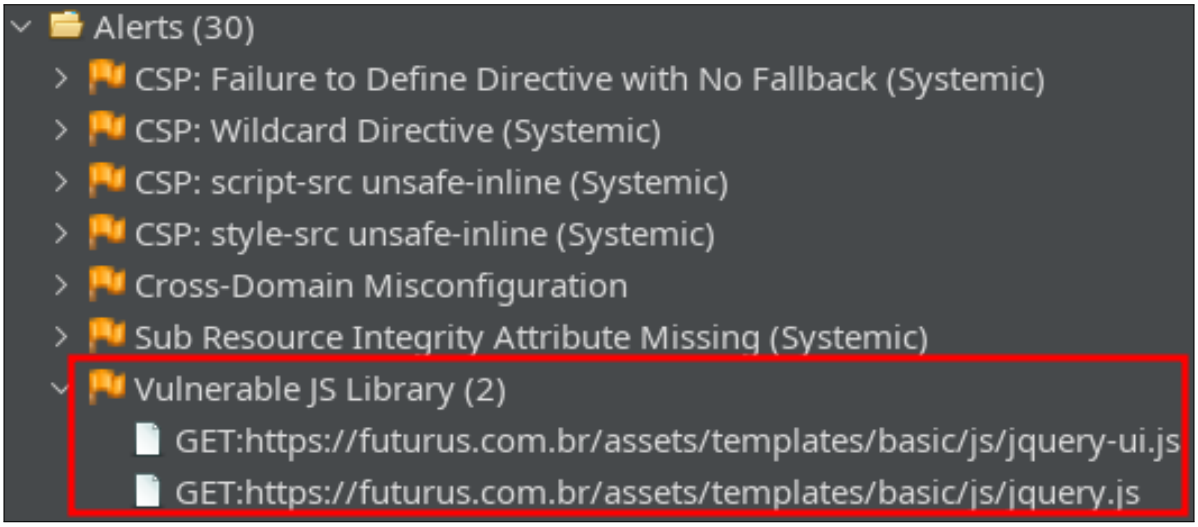



**Figure 3.3.:** Alerts triggered by the spider

- Look at the jquery-ui.js file under https://futurus.com.br/assets/templates/basic/js/jquery-ui.js:

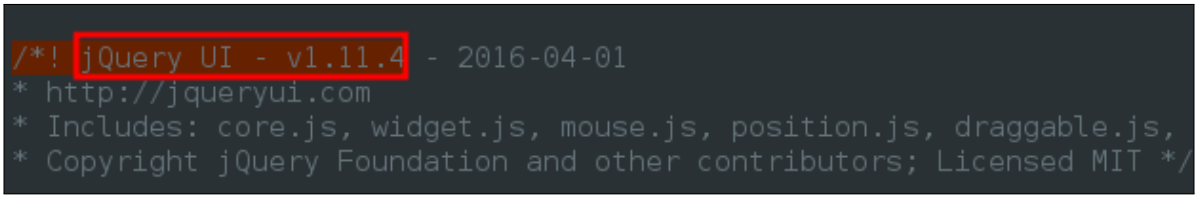



**Figure 3.4.:** jQuery Validation file

- Look at the jquery.js file under https://futurus.com.br/assets/templates/basic/js/jquery.js:

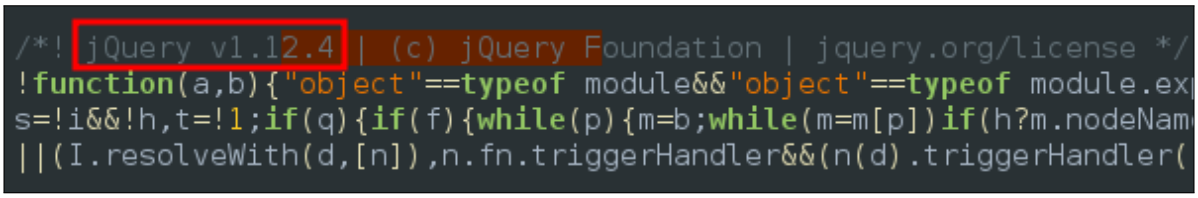



**Figure 3.5.:** jQuery Validation file

**Observed evidence** The scanned results were obtained using the OWASP ZAP tool.

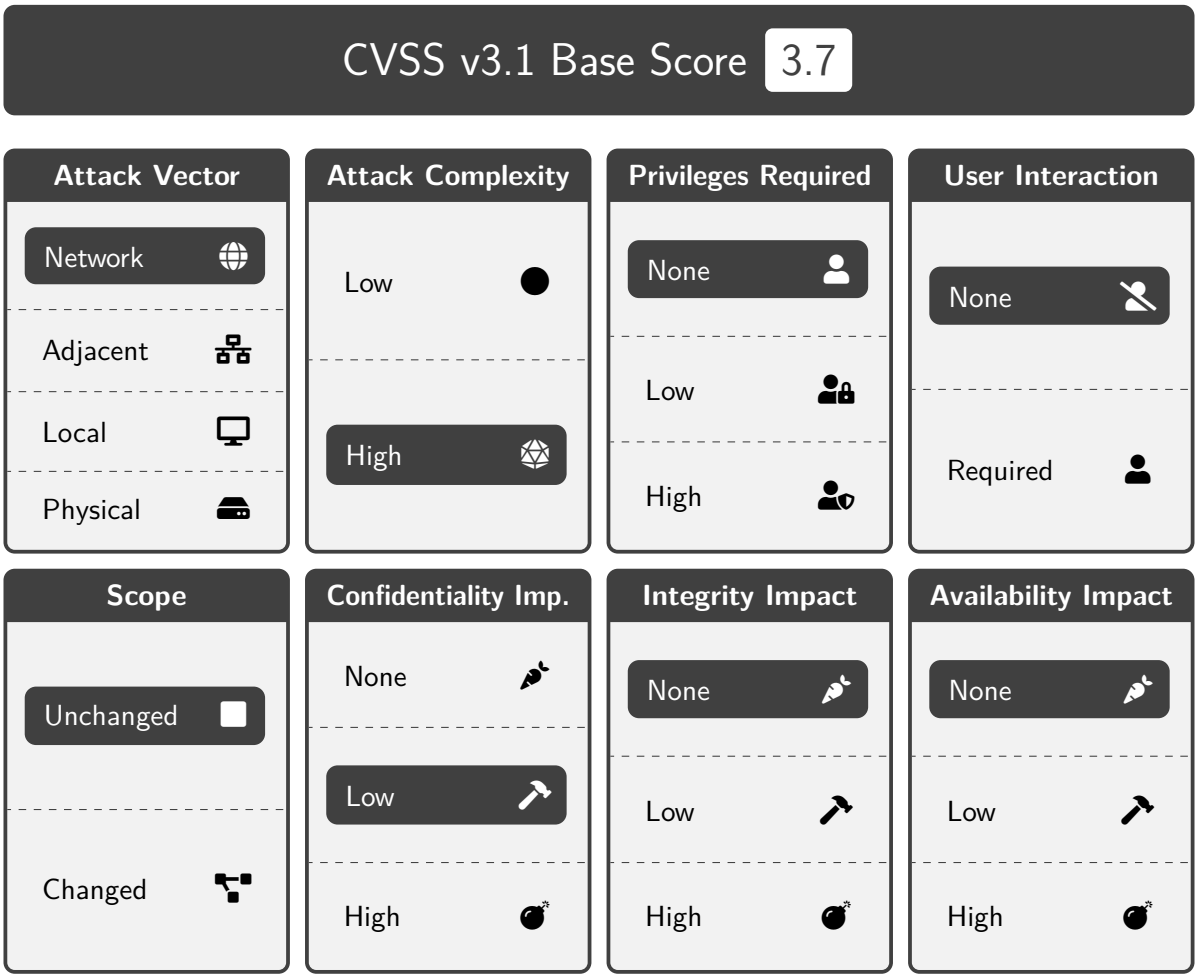
**Recommendations**

- Update jQuery and jQuery UI to supported versions.
- Review application code for unsafe use of DOM insertion, widget options, and other user-influenced client-side functionality.

**References**

- CWE MITRE definition
- OWASP Vulnerable and Outdated Component

## 3.2. LOW Weak Content Security Policy Due to script-src 'unsafe-inline'




**Summary** The application's Content Security Policy permits inline JavaScript execution through the use of 'unsafe-inline' in the script-src directive, which weakens CSP as a browser-side defense.

**Impact** This configuration reduces the effectiveness of CSP against cross-site scripting. If a separate injection issue exists, inline payloads may be easier to execute in the victim's browser.

### Proof of concept

- Define the scope:

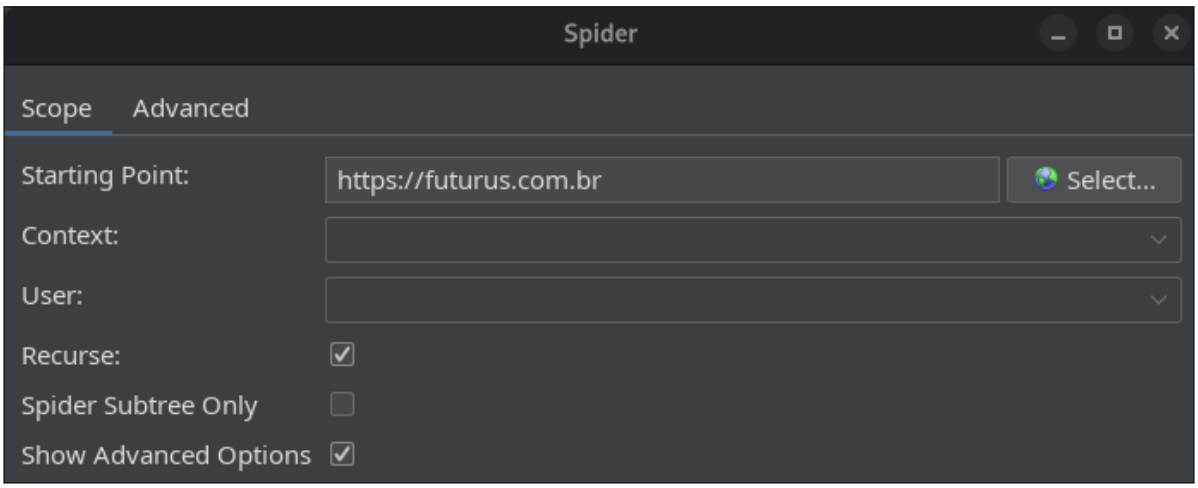



**Figure 3.6.:** Scope of the spider

- Set the following advanced settings:

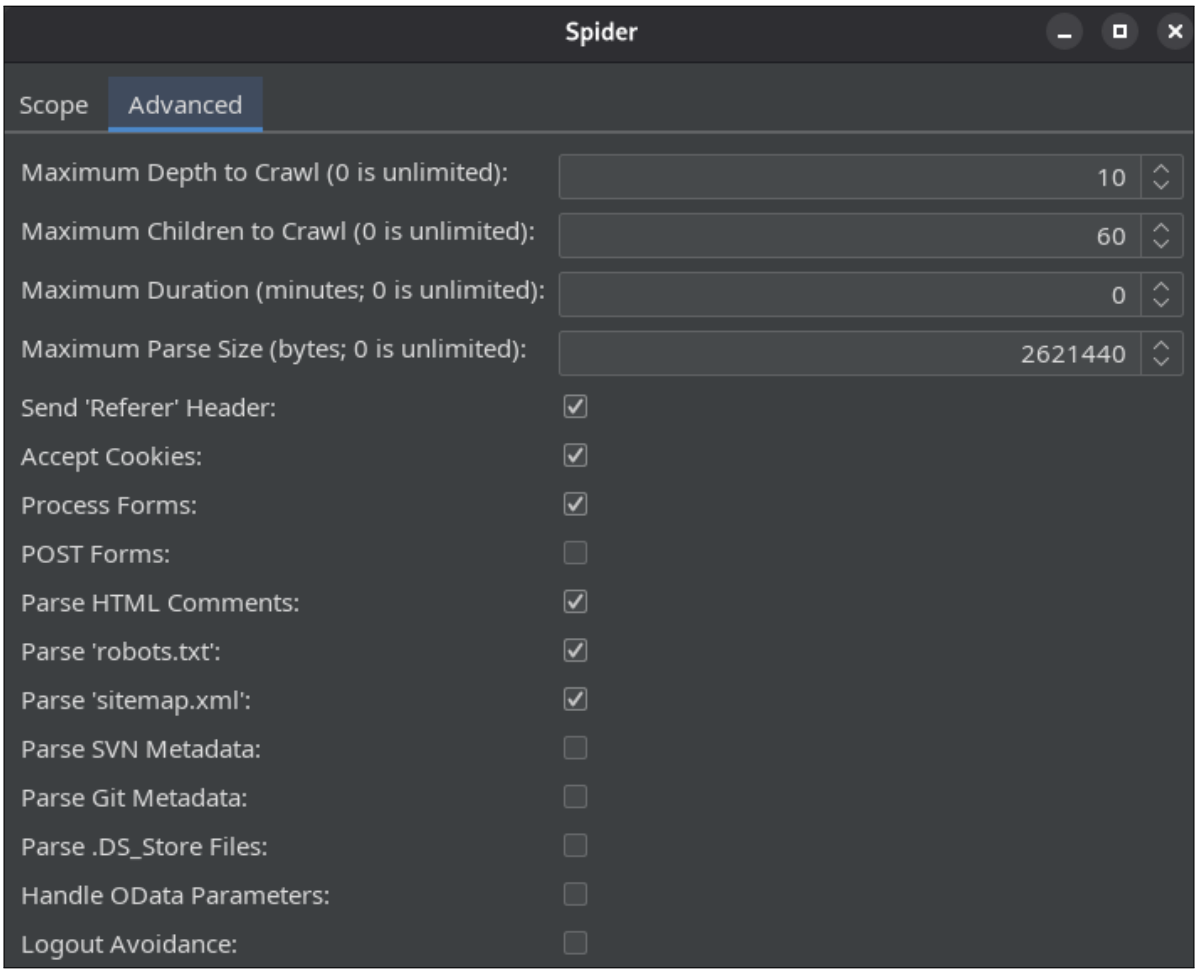



**Figure 3.7.:** Advanced settings to configure the spider

- Look the information gathered by the spider:

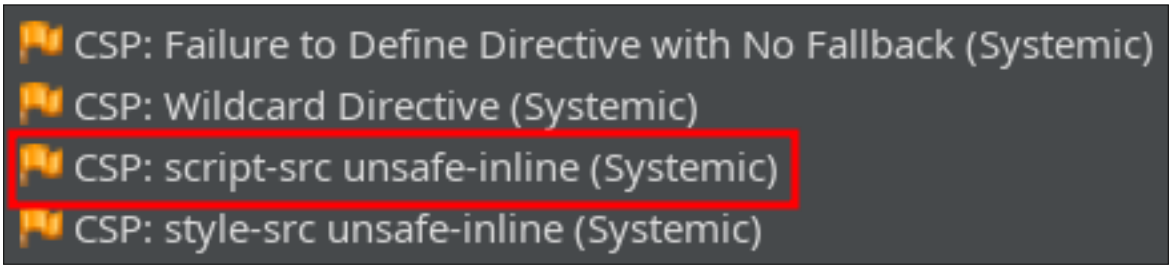



**Figure 3.8.:** Alert triggered by the spider

- Look at the affected files:

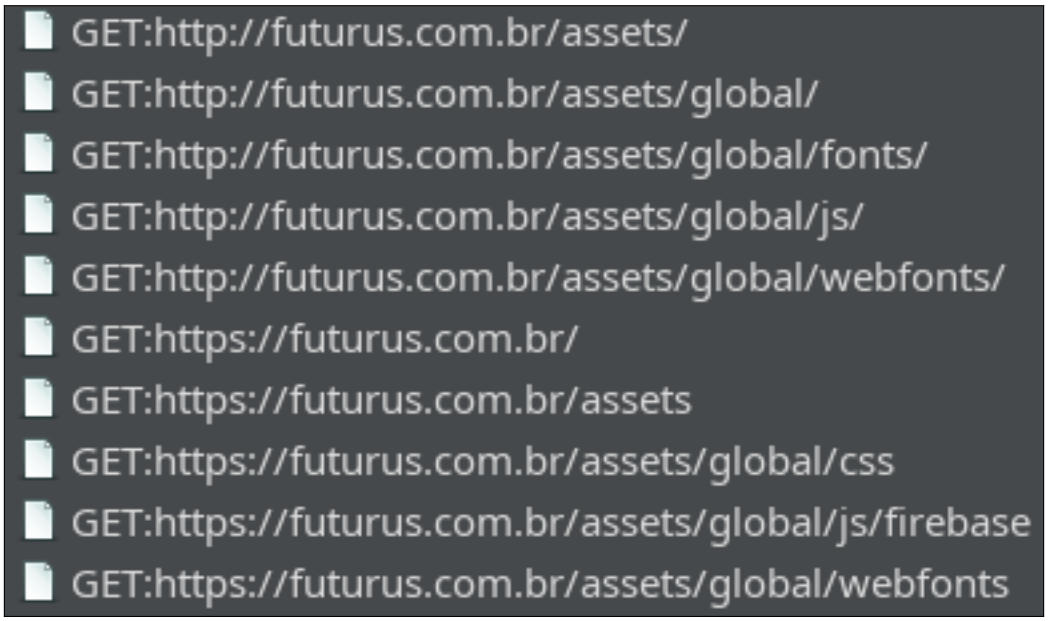



**Figure 3.9.:** Affected files

- ZAP general information and evidence:

| | |
|---|---|
| Parameter: | Content-Security-Policy |
| Attack: | |
| Evidence: | default-src 'self'; script-src 'self' 'unsafe-inline' https://cdn.jsdelivr.net https://cdnjs.cloudflare.com https://www.googletagmanager.com https://www.google-analytics.com https://www.gstatic.com https://*.firebaseio.com; style-src 'self' 'unsafe-inline' https://fonts.googleapis.com https://cdn.jsdelivr.net; font-src 'self' https://fonts.gstatic.com https://cdn.jsdelivr.net; img-src 'self' data: https: blob:; connect-src 'self' https://*.googleapis.com https://*.firebaseio.com wss://*.firebaseio.com https://firebaseinstallations.googleapis.com; frame-ancestors 'self'; |
| CWE ID: | 693 |

**Figure 3.10.:** Evidence Summary

**Observed evidence** The scanned results were obtained using the OWASP ZAP tool.

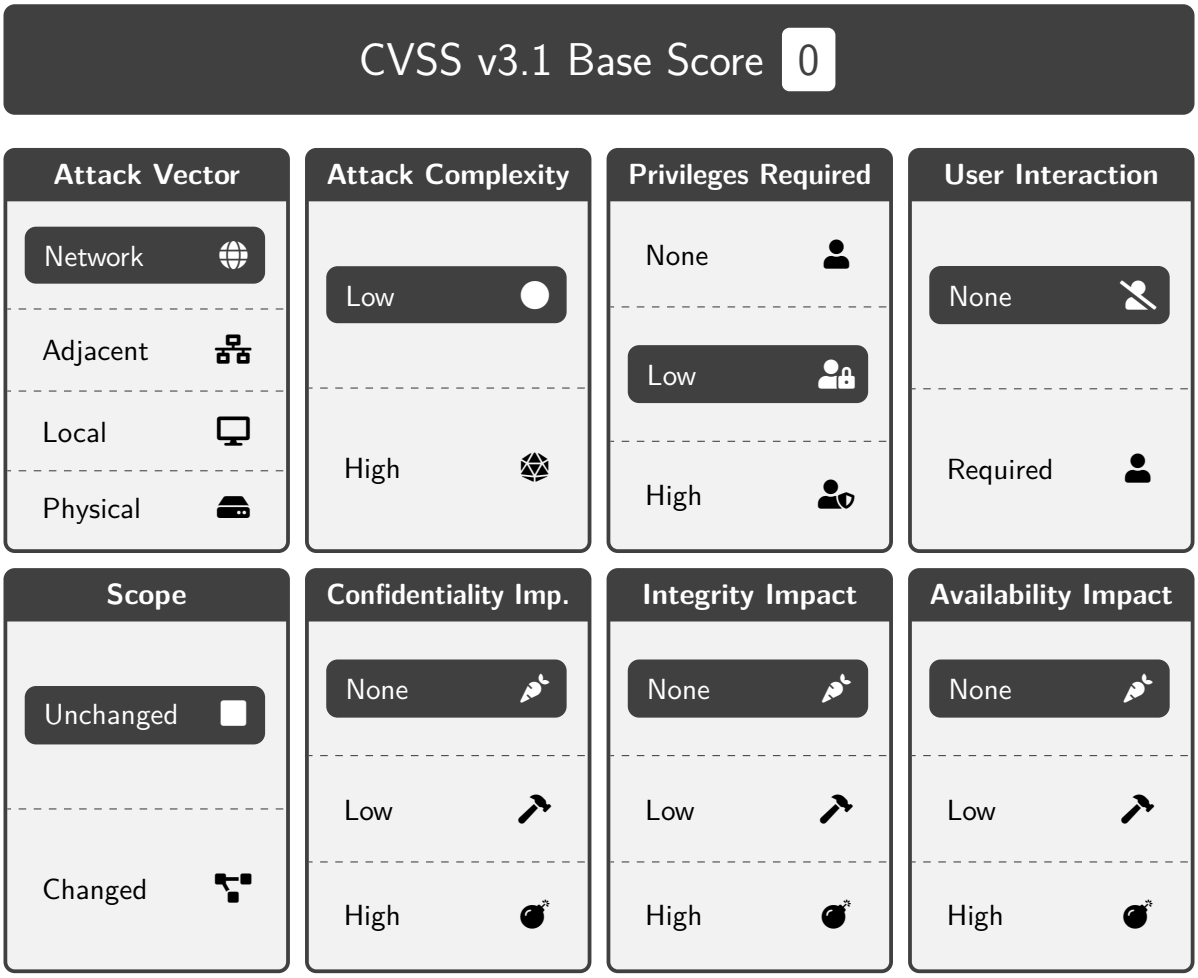
### Recommendations

- Remove 'unsafe-inline' from script-src.
- Replace inline scripts with external scripts, nonces, or hashes.

### References

- OWASP CSP Cheatsheet
- MITRE Definition

## 3.3. INFO Content Security Policy Hardening Gaps




**Summary** Additional CSP weaknesses were identified, including style-src 'unsafe-inline', an overly broad or insufficiently restricted img-src, and the absence of an explicit form-action directive. These issues reduce the strictness of the policy and represent defense-in-depth gaps.

**Impact** These weaknesses don't demonstrate exploitability by themselves, but they reduce the overall effectiveness of CSP and may increase the impact of future client-side injection or content-loading issues.

### Proof of concept

- Define the scope:

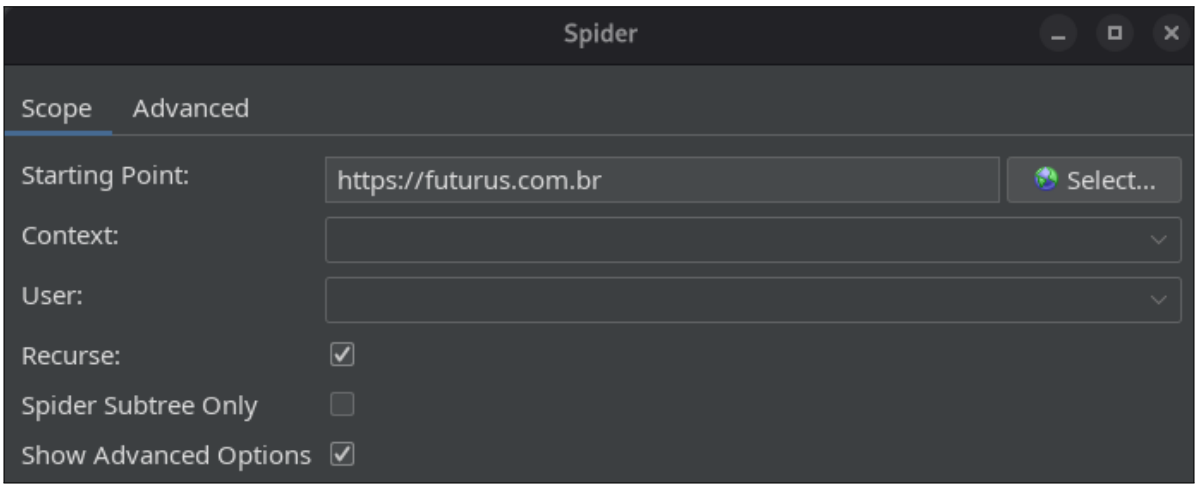



**Figure 3.11.:** Scope of the spider

- Set the following advanced settings:

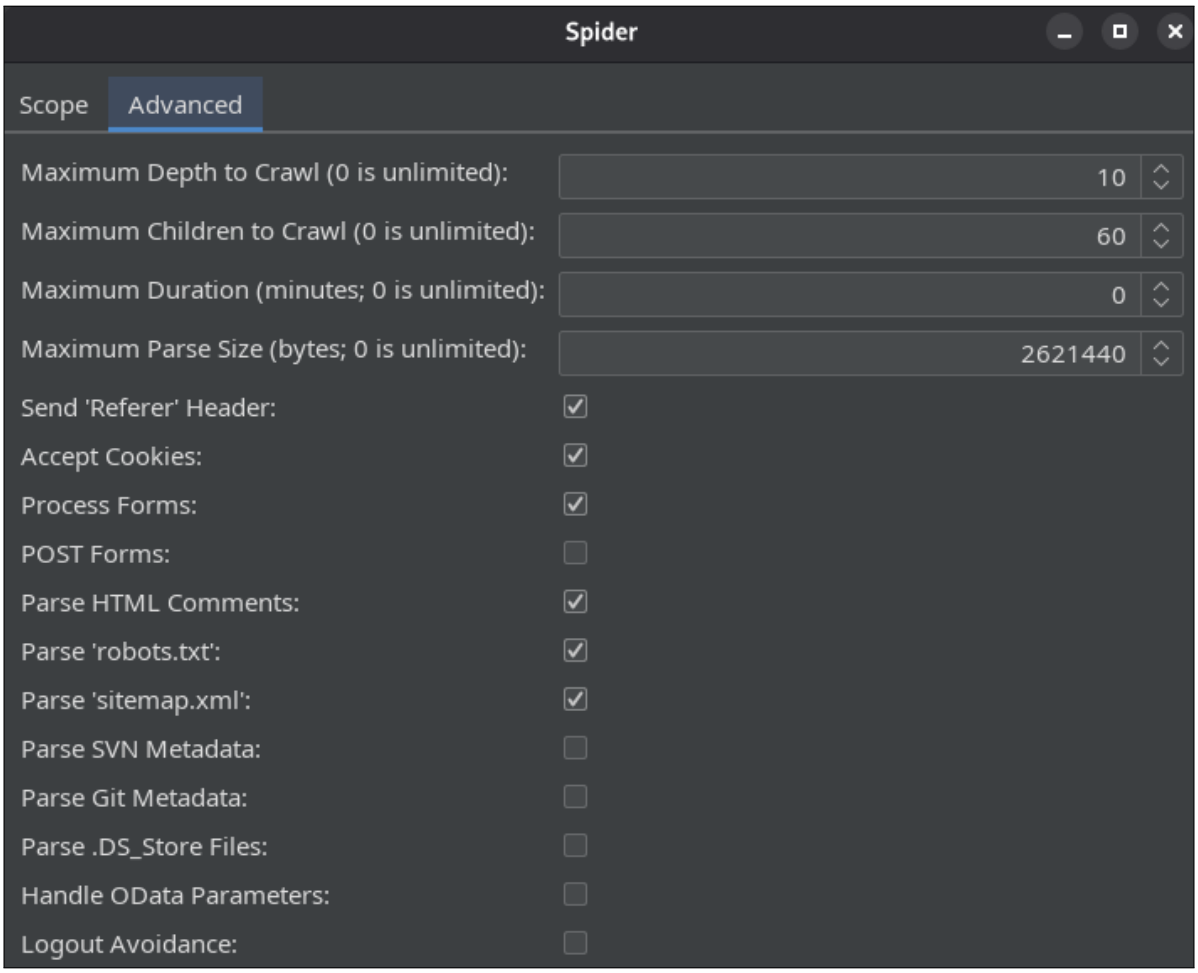



**Figure 3.12.:** Advanced settings to configure the spider

- Look the information gathered by the spider:

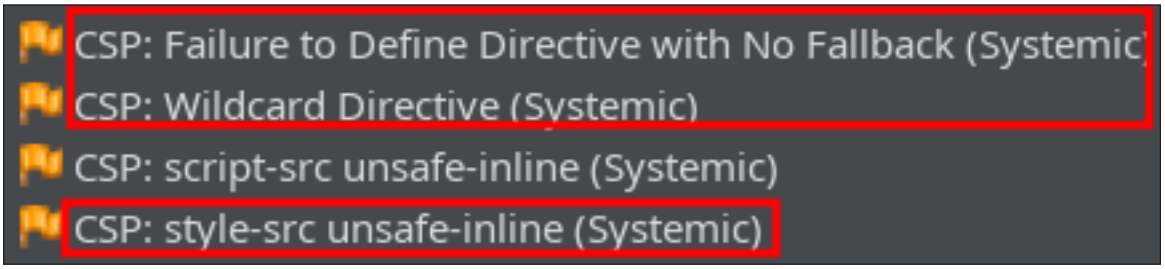



**Figure 3.13.:** Alerts triggered by the spider

- Look at the affected files:

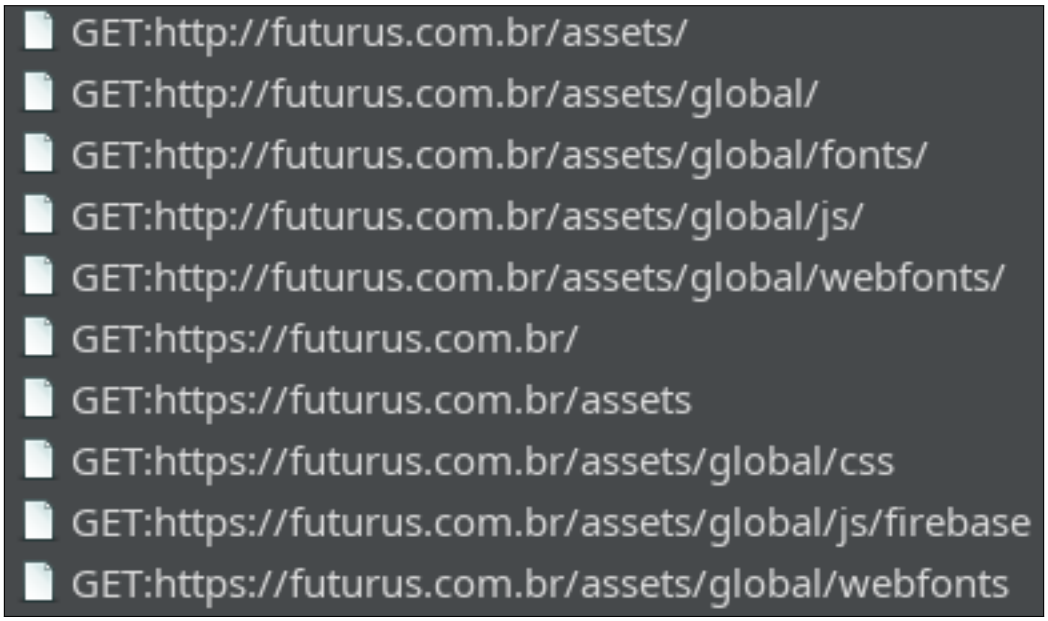



**Figure 3.14.:** Affected files

- ZAP general information and evidence:

| | |
|---|---|
| Parameter: | Content-Security-Policy |
| Attack: | |
| Evidence: | default-src 'self'; script-src 'self' 'unsafe-inline' https://cdn.jsdelivr.net https://cdnjs.cloudflare.com https://www.googletagmanager.com https://www.google-analytics.com https://www.gstatic.com https://*.firebaseio.com; style-src 'self' 'unsafe-inline' https://fonts.googleapis.com https://cdn.jsdelivr.net; font-src 'self' https://fonts.gstatic.com https://cdn.jsdelivr.net; img-src 'self' data: https: blob:; connect-src 'self' https://*.googleapis.com https://*.firebaseio.com wss://*.firebaseio.com https://firebaseinstallations.googleapis.com; frame-ancestors 'self'; |
| CWE ID: | 693 |

**Figure 3.15.:** Evidence Summary

**Observed evidence** The scanned results were obtained using the OWASP ZAP tool.

### Recommendations

- Restrict style-src and img-src to trusted sources only.
- Explicitly define form-action to approved destinations.

### References

- OWASP CSP Cheatsheet
- MITRE Definition

# 4. Remediation and Closure

## 4.1. Assessment Artifacts and Cleanup

| | |
|---|---|
| **Username** | test01 and test02. |

## 4.2. Mitigation and Patching Guidance

For each finding, the final report states the recommended fix, and displays a note about the vulnerability via the observed evidence field.

| | |
|---|---|
| **Priority 1** | Upgrade outdated third-party libraries to supported versions. |
| **Priority 2** | Harden the Content Security Policy header in accordance with the recommendations provided. |

## 4.3. Retest Recommendation

After remediation is complete, conduct a focused retest to verify that the reported weaknesses are no longer exploitable and that no regressions or compensating-control gaps remain.

## 4.4. Confidential Information Handling

**Confidential Data Handling Assurance**

All credentials, exports, screenshots, packet captures, and other confidential client data collected during the engagement must be securely deleted from analyst workstations, collaboration tools, and backups after report acceptance, except where a written retention requirement has been agreed with the client.

# A. Appendices

## A.1. Summary of Passing Tests

| Vulnerability Category | Testing Scope & Execution | Status |
|---|---|---|
| Vulnerable JavaScript Library | Scanned the jQuery Validation Plugin version. | Tested - Pass |
| Information Disclosure | Tested automated user enumeration. | Tested - Pass |

**Table A.1.:** Security Controls & Negative Findings Matrix